



Programação Web

ORM, Query Builders e Scope

Prof. Diego Cardoso Borda Castro



Organização de código

- Camadas de abstração
 - Index
 - Controller
 - Router
 - DTO
 - Service
 - Repositório
 - Entidade



Acesso ao banco

- Manipulação de dados em banco
 - Query pura
 - MySQL2
 - Query Builder
 - Maior flexibilidade
 - ORM
 - Organização
 - Entidades

	Gerência	Abstração	Complexidade
Raw	Alto		Baixo
Query builder	Baixo	Baixo	Baixo
ORM	Baixo	Alto	Média



Drizzle ORM

- <https://orm.drizzle.team/docs>
- Acesso ao banco para TypeScript
 - Query builder
 - você monta a query em SQL
 - ORM
 - mapeia tabelas em objetos tipados
 - não gera código; é só uma dependência



Drizzle ORM

- Query builder x ORM
 - Query builder (Knex)
 - controla o SQL, sem tipos
 - ORM (Sequelize)
 - abstrai o SQL em models
 - Drizzle fica no meio:
 - escreve parecido com SQL
 - com tipos e autocomplete vindos do schema



Drizzle ORM

- NPM init
- `npm i drizzle-orm better-sqlite3 express`
- `npm i -D drizzle-kit tsx typescript @types/node @types/express @types/better-sqlite3`
 - drizzle-orm
 - a lib que roda as queries
 - drizzle-kit
 - CLI que gera as migrations a partir do schema



Drizzle ORM

- Ts config é o arquivo que configura tudo do seu projeto ts
- Incluir o drizzle serve para adicionar ele para a IDE funcionar com o CLI

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "types": ["node"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src", "drizzle.config.ts"]
}
```



Drizzle ORM

- schema
 - onde estão as tabelas
- out
 - pasta das migrations
- dialect
 - 'sqlite'
- dbCredentials.url
 - o arquivo .db

```
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
  schema: './src/db/schema.ts',
  out: './drizzle',
  dialect: 'sqlite',
  casing: 'snake_case',
});
```

drizzle.config.ts



Drizzle ORM

- As duas APIs de consulta
 - **SQL-like**
 - `select().from().where()...`
 - Espelha o SQL, na mesma ordem
 - **Relational**
 - `db.query.users.findMany()`
 - traz dados aninhados, sem escrever JOIN



Drizzle ORM

```
buscarPorId: (id: number) =>
  db.query.cursos.findFirst({
    where: (c, ops) => ops.eq(c.id, id),
    with: {
      matriculas: {
        columns: { status: true, nota: true },
        with: { usuario: { columns: { id: true, nome: true } } },
      },
    },
  }),
```



```
{
  "id": 1,
  "titulo": "Programação Web",
  "cargaHoraria": 80,
  "ativo": true,
  "matriculas": [
    { "status": "ativa", "nota": null, "usuario": { "id": 1, "nome": "Ana Souza" } },
    { "status": "trancada", "nota": null, "usuario": { "id": 2, "nome": "Bruno Lima" } }
  ]
}
```



Drizzle ORM

```
buscarPorId: async (id: number) => {  
  const linhas = await db  
    .select({  
      curso: cursos,  
      status: matriculas.status,  
      nota: matriculas.nota,  
      usuario: { id: usuarios.id, nome: usuarios.nome },  
    })  
    .from(cursos)  
    .leftJoin(matriculas, eq(matriculas.cursoId, cursos.id))  
    .leftJoin(usuarios, eq(usuarios.id, matriculas.usuarioId))  
    .where(eq(cursos.id, id));  
}
```



```
[  
  { "curso": { "id": 1, "titulo": "Programação Web", ... }, "status": "ativa", "usuario": { "id": 1, "nome": "Ana Souza" } },  
  { "curso": { "id": 1, "titulo": "Programação Web", ... }, "status": "trancada", "usuario": { "id": 2, "nome": "Bruno Lima" } }  
]
```



Drizzle ORM

```
const sqlite = new Database(  
  process.env.DB_FILE ?? 'data/app.db'  
);  
sqlite.pragma('foreign_keys = ON');  
sqlite.pragma('journal_mode = WAL');  
  
export const db = drizzle(sqlite, {  
  schema,  
  logger: process.env.DB_LOG === '1',  
});
```

```
const pool = new Pool({  
  host: 'localhost',  
  port: 5432,  
  user: 'postgres',  
  password: 'postgres',  
  database: 'escola',  
});  
  
export const db = drizzle(pool, {  
  schema,  
  logger: process.env.DB_LOG === '1',  
});
```



Drizzle ORM

- DB
 - A instância do Drizzle que roda as queries
- Dialect
 - O banco alvo (sqlite, postgresql...)
- Driver
 - Pacote que fala com o banco
- Client
 - A conexão aberta pelo driver
- Schema
 - As tabelas descritas em TS

```
▼ drizzle
  > meta
  0000_curvy_praxagora.sql
  > node_modules
▼ src
  > cursos
  ▼ db
    TS migrate.ts
    TS schema.ts
    TS seed.ts
  > matriculas
  > usuarios
  TS server.ts
  .gitignore
  TS drizzle.config.ts
  {} package-lock.json
  {} package.json
  ⓘ README.md
  TS tsconfig.json
```



Schema

- Schema
 - Criação das tabelas
 - SqliteTable('nome', { colunas }, função)
 - pgtable
 - ...
 - Função
 - Index
 - PrimaryKey
 - ForeignKey
 - Check

```
export const usuariosCursos = sqliteTable('usuarios_cursos', {  
  usuarioId: integer('usuario_id').notNull().references(() => usuarios.id),  
  cursoId: integer('curso_id').notNull().references(() => cursos.id),  
}, (t) => [  
  primaryKey({ columns: [t.usuarioId, t.cursoId] }),  
]);
```



Schema





Schema

```
export const usuarios = sqliteTable('usuarios', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  nome: text('nome').notNull(),
  email: text('email').notNull().unique(),
  criadoEm: integer('criado_em', { mode: 'timestamp' })
    .notNull()
    .$defaultFn(() => new Date()),
});

export const usuariosRelations = relations(usuarios, ({ many }) => ({
  matriculas: many(matriculas),
}));
```



Schema

- References
 - Fica na coluna que guarda a chave estrangeira

```
export const matriculas = sqliteTable({
  'matriculas',
  {
    id: integer('id').primaryKey({ autoIncrement: true }),
    usuarioId: integer('usuario_id')
      .notNull()
      .references(() => usuarios.id, { onDelete: 'cascade' }),
    cursoId: integer('curso_id')
      .notNull()
      .references(() => cursos.id, { onDelete: 'cascade' }),
    status: text('status', { enum: ['ativa', 'trancada', 'concluida'] })
      .notNull()
      .default('ativa'),
    nota: real('nota'),
    matriculadoEm: integer('matriculado_em', { mode: 'timestamp' })
      .notNull()
      .$defaultFn(() => new Date()),
  },
  (t) => [
    uniqueIndex('matriculas_usuario_curso_idx').on(t.usuarioId, t.cursoId),
    index('matriculas_curso_idx').on(t.cursoId),
  ],
});
```



Schema

```
export const matriculasRelations = relations(matriculas, ({ one }) => ({
  usuario: one(usuarios, {
    fields: [matriculas.usuarioId],
    references: [usuarios.id],
  }),
  curso: one(cursos, {
    fields: [matriculas.cursoId],
    references: [cursos.id],
  }),
}));
```



Schema

cuidado com import circular ($A \leftrightarrow B$): junte num arquivo só

Tipo	Exemplo	.references()	relations()
1:N	usuário → matrículas	FK no lado N (matriculas.usuarioid)	many no lado 1 · one no lado N
1:1	usuário → perfil	FK no lado filho + .unique()	one nos dois lados
N:N	usuários ↔ cursos (via matriculas)	2 FKs na tabela de junção	many nos 2 lados + one×2 na junção
auto-relação	usuário → gerente	FK aponta pra própria tabela	one + many com relationName



Schema

- Ts
 - Tipado
 - Inferência de tipo

```
export type Curso = typeof cursos.$inferSelect;  
export type NovoCurso = typeof cursos.$inferInsert;
```

```
criar: (dados: typeof cursos.$inferInsert) =>  
  db  
    .insert(cursos)  
    .values(dados)  
    .returning()  
    .then((linhas) => linhas[0!]),
```

```
criar: (dados: NovoCurso) =>  
  db  
    .insert(cursos)  
    .values(dados)  
    .returning()  
    .then((linhas) => linhas[0!]),
```



Schema

- Coluna criadas automaticamente

```
export const cursos = sqliteTable('cursos', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  titulo: text('titulo').notNull().unique(),
  cargaHoraria: integer('carga_horaria').notNull().default(40),
  ativo: integer('ativo', { mode: 'boolean' }).notNull().default(true),
  resumo: text('resumo').generatedAlwaysAs(
    (): SQL => sql`${cursos.titulo} || ' (' || ${cursos.cargaHoraria} || 'h)'`,
    { mode: 'stored' },
  ),
  prerequisiteId: integer('prerequisite_id').references(
    (): AnySQLiteColumn => cursos.id,
  ),
});
```



Schema

- Auto Relacionamento
- Self Join

```
export const cursosRelations = relations(cursos, ({ many, one }) => ({
  matriculas: many(matriculas),

  prerequisite: one(cursos, {
    fields: [cursos.prerequisiteId],
    references: [cursos.id],
    relationName: 'prerequisite',
  }),

  cursosQueExigem: many(cursos, {
    relationName: 'prerequisite',
  }),
}));
```



Schema

- Schema em vários arquivos
 - divida por entidade: users.ts, posts.ts...
 - um index.ts (barrel) re-exporta tudo

```
export * from '../usuarios/usuario.schema';  
export * from '../cursos/curso.schema';  
export * from '../matriculas/matricula.schema';
```

```
export const db = drizzle(sqlite, {  
  schema,  
  logger: process.env.DB_LOG === '1',  
});
```

```
export default defineConfig({  
  schema: './src/db/schema.ts',  
  out: './drizzle',  
  dialect: 'sqlite',  
  casing: 'snake_case',  
  verbose: true,  
  strict: true,  
});
```



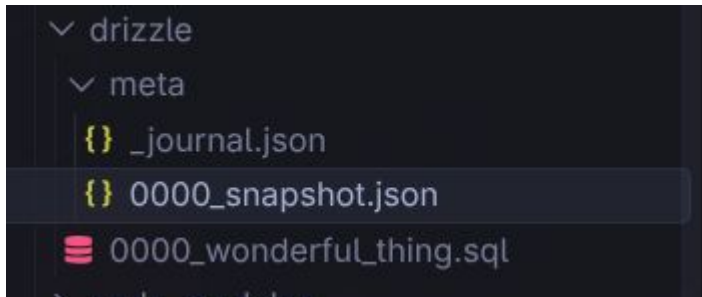
Migration

- Um passo versionado de mudança no banco
- Cada alteração de schema vira um arquivo
- Criar tabela, adicionar coluna, criar índice...
- Fica no git
 - Todo mundo aplica igual
- O banco 'anda junto' com o código
 - compara o schema com o último snapshot e escreve o diff em SQL
 - gera .sql + _journal.json + snapshot (foto do schema)



Migration

- npx drizzl-kit migrate
 - Filosofia é forward-only
 - Não tem Down e up



```
import { migrate } from 'drizzle-orm/better-sqlite3/migrator';
import { db } from './db';

migrate(db, { migrationsFolder: './drizzle' });
```



```
$ npx drizzle-kit generate  
  
drizzle/  
  0000_init.sql  
meta/  
  _journal.json  
  0000_snapshot.json
```





- drizzle-kit — outros comandos

```
● ● ●  
$ npx drizzle-kit drop      # remove uma migration do historico  
  
$ npx drizzle-kit check    # detecta colisao entre migrations  
  
$ npx drizzle-kit up       # atualiza o formato dos snapshots  
  
$ npx drizzle-kit export   # imprime o SQL do schema
```



```
$ npx drizzle-kit pull
```

```
-> src/db/schema.ts
```

```
-> src/db/relations.ts
```





- No nosso projeto
 - `src/db/seed.ts` — conexão própria, não sobe o server
 - `onConflictDoNothing()` deixa repetível

src/db/seed.ts

```
await db.insert(usuarios).values([
  { nome: 'Ana Souza', email: 'ana@escola.dev' },
  { nome: 'Bruno Lima', email: 'bruno@escola.dev' },
]).onConflictDoNothing();

await db.insert(cursos).values([
  { titulo: 'Programação Web', cargaHoraria: 80 },
  { titulo: 'Banco de Dados', cargaHoraria: 60 },
]).onConflictDoNothing();
```





usuario.repository.ts

```
listar: (nome?: string) =>
  db.select().from(usuarios)
    .where(nome
      ? like(usuarios.nome, `%${nome}%`)
      : undefined)
    .orderBy(usuarios.nome),
```



usuario.repository.ts

```
criar: (dados: typeof usuarios.$inferInsert) =>  
  db.insert(usuarios)  
    .values(dados)  
    .returning()  
    .then((linhas) => linhas[0]!),
```



usuario.repository.ts



```
atualizar: (id: number, dados: Partial<...>) =>  
  db.update(usuarios)  
    .set(dados)  
    .where(eq(usuarios.id, id))  
    .returning()  
    .then((linhas) => linhas.at(0)),
```



usuario.repository.ts

```
remover: (id: number) =>  
  db.delete(usuarios)  
    .where(eq(usuarios.id, id))  
    .returning()  
    .then((linhas) => linhas.at(0)),
```



- Join
 - innerJoin liga usuarios e cursos
 - resultado vem achatado no select

matricula.repository.ts

```
listar: () =>
  db.select({
    aluno: usuarios.nome,
    curso: cursos.titulo,
    status: matriculas.status,
  })
  .from(matriculas)
  .innerJoin(usuarios, eq(matriculas.usuarioId, usuarios.id))
  .innerJoin(cursos, eq(matriculas.cursoId, cursos.id)),
```



usuario.repository.ts

```

    buscarPorId: (id: number) =>
      db.query.usuarios.findFirst({
        where: (u, ops) => ops.eq(u.id, id),
        with: {
          matriculas: {
            with: { curso: true },
          },
        },
      }),

```





- No nosso projeto
 - matricular(): valida e insere na mesma transação

matricula.service.ts

```
matricular: (usuarioId: number, cursoId: number) =>
  db.transaction((tx) => {
    const curso = tx.select().from(cursos)
      .where(eq(cursos.id, cursoId)).get();
    if (!curso) throw new Error('curso nao encontrado');
    if (!curso.ativo) throw new Error('curso inativo');

    return tx.insert(matriculas)
      .values({ usuarioId, cursoId })
      .returning().get();
  }),
```